

Universidad Nacional de Ingeniería
Facultad de Ingeniería Eléctrica y Electrónica



Aprendizaje de Máquina y Minería de Datos

Big Data: Real-Time Retrieval

GRUPO 5

Integrantes:

Cespedes Gonzales, Rodrigo Arturo - 20150357F

Luna Chaysa, Nelson - 20171375C

Tiburcio Huamani, Marcelo Benjamín - 20182138H

Docente:

Mg. Ing. Yury Tello Canchapoma

LIMA - PERÚ

2026

ÍNDICE DE CONTENIDOS

Resumen	3
1. Introducción	4
2. El Paradigma del Tiempo Real	5
2.1 Definición y Características	5
2.2 Diferencia con el Procesamiento Batch	6
2.3 SLAs y Métricas de Rendimiento	6
2.4 Desafíos Técnicos	7
3. Arquitectura de Recuperación Rápida	8
3.1 Visión General del Sistema	8
3.2 Componentes del Ecosistema Tecnológico	9
4. Apache HBase — Almacenamiento NoSQL Distribuido	10
4.1 Introducción a HBase	10
4.2 Arquitectura Interna	11
4.3 Modelo de Datos Multidimensional	13
4.4 Diseño del RowKey	14
4.5 Mecanismos de Rendimiento: Compaction y Split	16
5. Apache Elasticsearch — Motor de Búsqueda Compleja	17
5.1 Introducción y Características	17
5.2 Arquitectura y Conceptos Clave	18
5.3 El Índice Invertido (Inverted Index)	19
5.4 Análisis de Texto y Tokenización	20
5.5 Optimización del Rendimiento	21
6. Arquitectura Combinada: HBase + Elasticsearch	22
6.1 Secondary Indexing	22
6.2 Flujo de Ingesta y Consulta	23
7. GES — Graph Engine Service	24
7.1 Introducción al Análisis de Grafos	24
7.2 Arquitectura de GES	25
8. Casos de Uso Empresariales	26
8.1 Sector Financiero	26
8.2 E-Commerce y Retail	27
8.3 Ciberseguridad y Logs	27
8.4 Telco y Call Centers	28
9. Conclusiones	29
10. Referencias Bibliográficas	30

RESUMEN

La proliferación de datos a escala de Petabytes ha transformado radicalmente las exigencias sobre los sistemas de información. Industrias como la financiera, el comercio electrónico y la ciberseguridad requieren recuperar registros específicos de almacenes masivos en tiempos de respuesta de milisegundos. El procesamiento offline por lotes (batch) resulta insuficiente para satisfacer estos requerimientos de baja latencia y alta concurrencia.

La presente monografía analiza en profundidad las soluciones tecnológicas que conforman el estándar de la industria para la Búsqueda y Recuperación en Tiempo Real (Real-Time Retrieval) en ecosistemas Big Data. Se estudia Apache HBase como motor de almacenamiento NoSQL distribuido de acceso aleatorio de alta velocidad, Apache Elasticsearch como motor de búsqueda e indexación de texto completo, y GES (Graph Engine Service) para el análisis de redes complejas de relaciones. Se examina asimismo la sinergia arquitectónica entre estas herramientas, el patrón de Secondary Indexing, y su aplicación en casos de uso empresariales reales.

Los hallazgos demuestran que el rendimiento en milisegundos no es resultado de la velocidad del hardware, sino de un diseño arquitectónico riguroso: el esquema del RowKey en HBase, la estrategia de sharding en Elasticsearch, y el uso intensivo de la memoria RAM como capa intermedia de caché.

Palabras clave: Big Data, Real-Time Retrieval, HBase, Elasticsearch, NoSQL, Índice Invertido, LSM-Tree, Secondary Indexing, GES, Baja Latencia.

1. INTRODUCCIÓN

El volumen de datos generados por la humanidad crece a un ritmo exponencial. Según estimaciones de la industria, en 2025 el mundo genera más de 120 Zettabytes de datos anuales, procedentes de redes sociales, sensores IoT, transacciones financieras, logs de sistemas y registros médicos, entre otros. Este fenómeno, conocido como Big Data, se caracteriza por las denominadas '5 Vs': Volumen, Velocidad, Variedad, Veracidad y Valor.

Ante esta realidad, las organizaciones se enfrentan a un desafío crítico: no basta con almacenar grandes volúmenes de datos; es imperativo poder acceder a ellos de forma instantánea. Un banco que tarda 10 segundos en verificar el historial de crédito de un cliente pierde una oportunidad de negocio. Un sistema de detección de fraude que responde en 5 segundos llega tarde para bloquear una transacción fraudulenta. Esta necesidad de acceso instantáneo a datos masivos define el campo de la Recuperación en Tiempo Real o Real-Time Retrieval.

Históricamente, las bases de datos relacionales (RDBMS) como Oracle o MySQL ofrecían acceso rápido, pero no escalaban a volúmenes de Petabytes. El ecosistema Hadoop, por su parte, fue diseñado para procesamiento secuencial masivo (batch), resultando inadecuado para lecturas aleatorias de baja latencia. La solución emergió con las bases de datos NoSQL y los motores de búsqueda distribuidos, que adoptaron estructuras de datos fundamentalmente distintas para resolver este problema.

Este documento examina el estado del arte en Real-Time Retrieval para entornos Big Data, con énfasis en las tecnologías Apache HBase, Apache Elasticsearch y GES, estudiando sus principios de diseño, arquitecturas internas y patrones de integración.

2. EL PARADIGMA DEL TIEMPO REAL

2.1 Definición y Características

La recuperación de datos en tiempo real (Real-Time Retrieval) se define como la capacidad de localizar y recuperar un registro específico —o un pequeño conjunto de registros— a partir de almacenes de datos a escala de Petabytes, con un tiempo de respuesta que oscila entre milisegundos y pocos segundos. El énfasis recae en la baja latencia de respuesta para consultas individuales (Point Lookup), en contraposición al procesamiento batch que maximiza el throughput agregado.

Las características fundamentales de un sistema de Real-Time Retrieval son: (1) Baja latencia: tiempos de respuesta inferiores a 1 segundo para consultas por clave primaria y menores a 3 segundos para búsquedas complejas; (2) Alta concurrencia: capacidad de atender simultáneamente miles de consultas por segundo (QPS) sin degradación del servicio; (3) Escalabilidad horizontal: crecimiento lineal de la capacidad al agregar nodos al clúster; (4) Alta disponibilidad: operación ininterrumpida mediante replicación y redundancia; (5) Consistencia eventual: sacrificio controlado de consistencia inmediata en favor de rendimiento y disponibilidad.

Soluciones Big Data: Recuperación de Datos en Tiempo Real (Real-Time Retrieval)

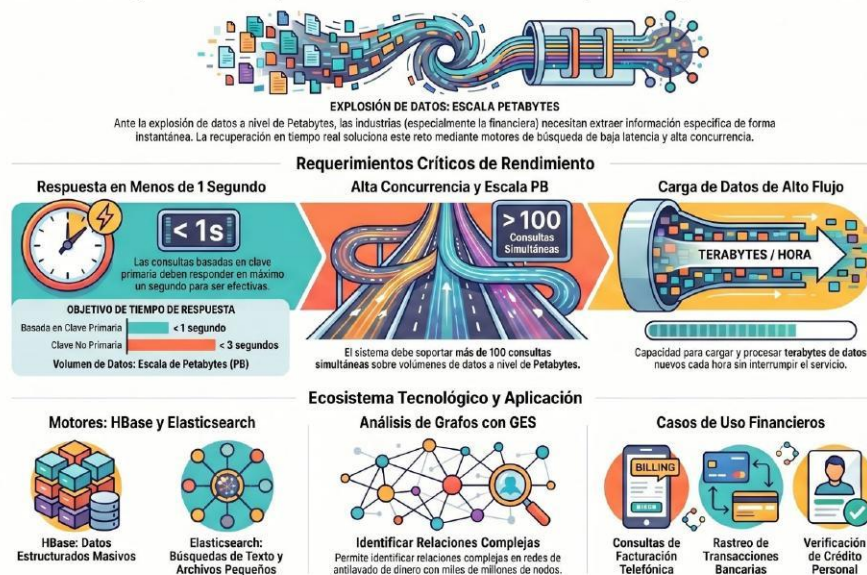


Figura 1. Requerimientos críticos de rendimiento en soluciones Big Data de recuperación en tiempo real. Fuente: Material del curso (Tello Canchapoma, 2025).

2.2 Diferencia con el Procesamiento Batch

El paradigma batch y el paradigma de tiempo real representan filosofías de diseño opuestas, cada una optimizada para un tipo diferente de carga de trabajo:

- **Procesamiento Batch (Offline):** Escanea la tabla completa (Full Table Scan) para calcular agregaciones masivas sobre millones o miles de millones de registros. El objetivo es maximizar el Throughput (rendimiento global), es decir, procesar la mayor cantidad de datos posible en un período dado. Herramientas paradigmáticas: MapReduce, Apache Hive, Apache Spark batch. La latencia de una consulta individual puede ser de minutos u horas.
- **Recuperación en Tiempo Real:** Realiza acceso aleatorio directo a datos puntuales (Point Lookup o Random Read). El objetivo es minimizar la Latency (latencia de respuesta individual), independientemente del volumen total de datos almacenados. Herramientas paradigmáticas: HBase, Elasticsearch, Redis. La latencia de una consulta individual debe ser de milisegundos.

La distinción clave reside en la estructura de datos subyacente: el procesamiento batch aprovecha las lecturas secuenciales en disco (eficientes para discos magnéticos), mientras que el Real-Time Retrieval requiere índices que permitan saltar directamente al registro deseado sin recorrer los datos intermedios.

2.3 SLAs y Métricas de Rendimiento

Los Acuerdos de Nivel de Servicio (Service Level Agreements – SLA) en sistemas de Real-Time Retrieval definen compromisos cuantitativos de rendimiento. Los SLAs más comunes en la industria son los siguientes:

Tipo de Consulta	SLA Objetivo	Herramienta Típica
Búsqueda por clave primaria	< 100 milisegundos	HBase
Búsqueda por atributos múltiples	< 3 segundos	Elasticsearch
Alta concurrencia (QPS)	> 1,000 consultas/segundo	HBase + Elasticsearch
Carga de datos nuevos	Terabytes por hora	Kafka + HBase

Tabla 1. SLAs comunes en sistemas de Real-Time Retrieval.

2.4 Desafíos Técnicos

Alcanzar los SLAs descritos sobre volúmenes de datos a escala de Petabytes implica superar barreras técnicas fundamentales:

- **El Problema del I/O Magnético:** Las bases de datos relacionales y HDFS tradicional están optimizados para lecturas secuenciales. Las lecturas aleatorias en discos magnéticos son extremadamente lentas debido al seek time mecánico (tiempo que el cabezal tarda en posicionarse). Un HDD tiene un seek time de 5-10 ms, lo que limita a ~100-200 lecturas aleatorias por segundo.

- El Problema de la Escala del Índice: Los árboles B+ (B-Tree), utilizados en MySQL y Oracle, son eficientes cuando el índice cabe completamente en memoria RAM. Cuando el índice supera la RAM disponible, cada consulta puede requerir múltiples accesos a disco, degradando drásticamente el rendimiento.
- El Problema de la Concurrencia: Atender miles de consultas simultáneas requiere arquitecturas que eviten cuellos de botella centralizados (hot spots) y distribuyan la carga uniformemente entre los nodos del clúster.
- El Problema del Esquema Flexible: En entornos Big Data, el esquema de los datos evoluciona constantemente. Las bases de datos relacionales imponen esquemas rígidos que dificultan la adaptación a nuevos tipos de datos.

La solución tecnológica que ha emergido combina bases de datos NoSQL con estructuras LSM-Tree (Log-Structured Merge-Tree) para escrituras rápidas y lecturas indexadas, junto con motores de búsqueda basados en índices invertidos distribuidos en memoria RAM. Esta combinación forma la base de la arquitectura moderna de Real-Time Retrieval.

3. ARQUITECTURA DE RECUPERACIÓN RÁPIDA

3.1 Visión General del Sistema

Una arquitectura moderna de Real-Time Retrieval en ecosistemas Big Data no es monolítica, sino que integra múltiples capas especializadas, cada una optimizada para una función específica. El flujo general de datos sigue el siguiente patrón: los datos se ingestan a través de sistemas de mensajería distribuida (como Apache Kafka), se almacenan en bases de datos NoSQL de alto rendimiento (HBase), se indexan simultáneamente en motores de búsqueda (Elasticsearch), y se sirven a los usuarios finales a través de APIs de baja latencia.

La separación entre la capa de almacenamiento y la capa de indexación es fundamental: permite que cada componente sea optimizado de forma independiente para su función específica. HBase se optimiza para almacenar y recuperar grandes volúmenes de datos estructurados mediante claves exactas, mientras que Elasticsearch se optimiza para búsquedas complejas de texto y atributos múltiples.



Solution

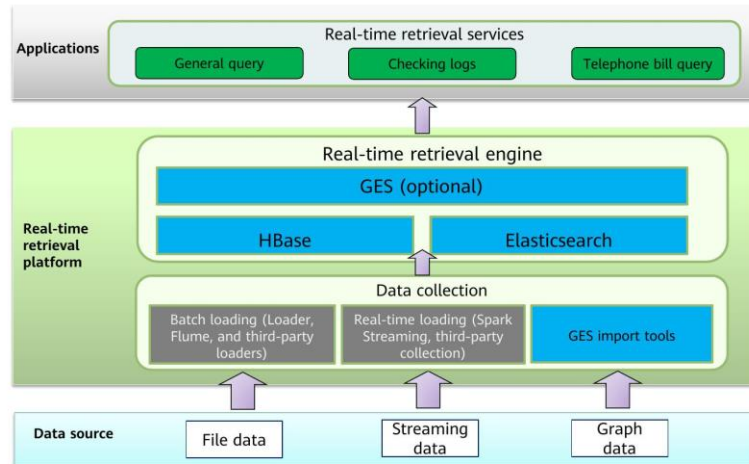


Figura 2. Arquitectura de la plataforma de Real-Time Retrieval. De abajo hacia arriba: fuentes de datos (archivos, streaming, grafos), capa de recolección (batch/streaming), motor de recuperación (HBase + Elasticsearch + GES opcional) y capa de aplicaciones (consultas generales, logs, facturación). Fuente: Capítulo 3, Huawei Technologies (2020).



Association and Position of Components in the Real-Time Retrieval Engine

	Primary Key Query	Non-Primary Key Query	Document Retrieval	Massive Data	Real-time Retrieval	Graph Querying and Analyzing
HBase	✓	MRS supports secondary indexes.		✓	✓	
Elasticsearch	✓	✓	✓	✓ The performance of massive data is inferior to that of HBase, and the cost-effectiveness is low.	✓ Real-time hot data	
GES				✓ Data is stored in HBase and Elasticsearch.	✓ Near-real-time query of complex graphs	✓

Figura 3. Posicionamiento y capacidades de los componentes del motor de recuperación en tiempo real. HBase, Elasticsearch y GES satisfacen diferentes tipos de consultas. Fuente: Capítulo 3, Huawei Technologies (2020).

3.2 Componentes del Ecosistema Tecnológico

El ecosistema tecnológico de Real-Time Retrieval en Big Data comprende tres motores principales, cada uno con capacidades y casos de uso diferenciados:

- Apache HBase: Motor principal de almacenamiento NoSQL para datos estructurados masivos. Sobresale en consultas por clave primaria (RowKey) con latencias de microsegundos a milisegundos. Ideal para acceso directo a registros individuales sobre tablas de miles de millones de filas.
- Apache Elasticsearch: Motor de búsqueda e indexación de texto completo. Sobresale en búsquedas por múltiples atributos, texto libre, búsqueda difusa y análisis geoespacial. Complementa a HBase proporcionando indexación secundaria sobre los datos almacenados.
- GES (Graph Engine Service): Motor de base de datos de grafos distribuida construida sobre HBase y Elasticsearch. Especializado en consultas de redes de relaciones complejas: detección de comunidades, cálculo de caminos más cortos, análisis de centralidad y detección de patrones en grafos de miles de millones de nodos.

La decisión sobre qué motor utilizar depende del tipo de consulta a resolver, no de preferencias tecnológicas. La combinación HBase + Elasticsearch cubre la mayoría de los casos de uso empresariales, mientras que GES se introduce cuando la naturaleza grafal de los datos es predominante.

4. APACHE HBASE — ALMACENAMIENTO NoSQL DISTRIBUIDO

4.1 Introducción a HBase

Apache HBase es una base de datos NoSQL distribuida, orientada a columnas, multidimensional, versionada y construida sobre el sistema de archivos distribuido HDFS (Hadoop Distributed File System). Fue diseñada como implementación open-source del paper de Google BigTable (Chang et al., 2006) y está gestionada por la Apache Software Foundation.

HBase proporciona acceso aleatorio de lectura y escritura en tiempo real a tablas gigantes, con capacidad para almacenar y gestionar tablas de miles de millones de filas y millones de columnas. Su propósito fundamental es resolver el problema que HDFS (optimizado para lectura secuencial) no puede resolver: el acceso aleatorio de alta velocidad a registros individuales en datasets masivos.

Las características diferenciadoras de HBase son:

- Alta confiabilidad y tolerancia a fallos: mediante replicación de datos en HDFS y mecanismos de recuperación automática.
- Alto rendimiento: estructuras de datos optimizadas para escrituras y lecturas aleatorias de alta velocidad.
- Orientación a columnas: el almacenamiento físico agrupa columnas relacionadas juntas, no filas completas, lo que permite proyecciones eficientes.
- Escalabilidad horizontal: el clúster puede expandirse añadiendo nodos sin interrupciones del servicio.

- Esquema disperso: las filas no necesitan tener valores para todas las columnas; las celdas vacías no ocupan espacio en disco.
- Versionado automático: cada celda puede almacenar múltiples versiones históricas del dato, distinguidas por timestamps.
- Dependencias clave: HDFS como almacenamiento físico subyacente con replicación de bloques; ZooKeeper para coordinación del clúster y gestión de estado distribuido.

4.2 Arquitectura Interna

La arquitectura de HBase es cliente-servidor con tres capas principales: el HMaster (coordinador), los RegionServers (trabajadores) y ZooKeeper (servicio de coordinación). Cada capa tiene responsabilidades claramente delimitadas.

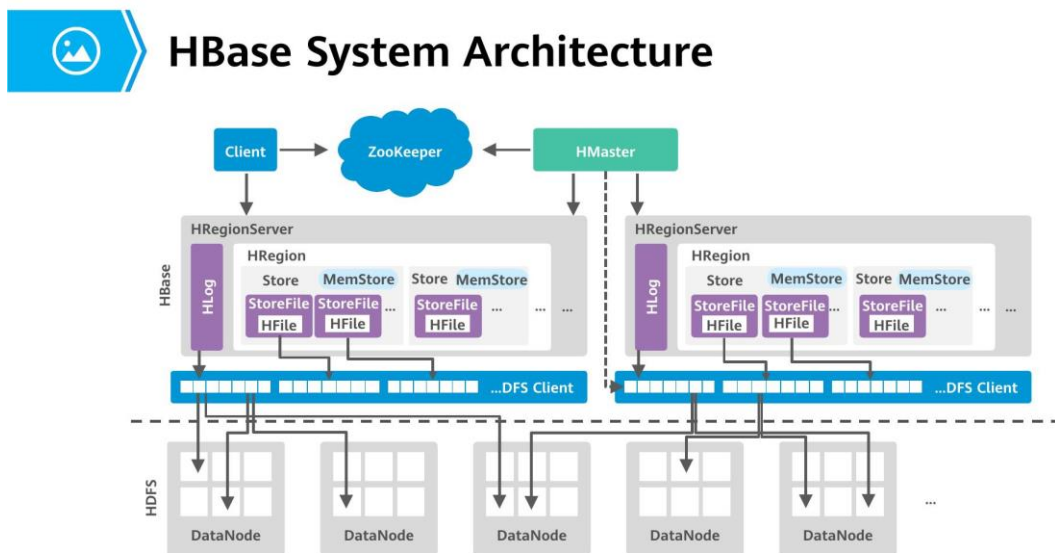


Figura 4. Arquitectura del sistema HBase: Client → ZooKeeper → HMaster; HRegionServers gestionando HRegions con Store, MemStore y StoreFiles (HFiles); integración con la capa HDFS (DataNodes). Fuente: Capítulo 3, Huawei Technologies (2020).

HMaster — El Coordinador

El HMaster es el nodo maestro del clúster HBase. Sus responsabilidades son exclusivamente administrativas y de coordinación; no participa en el flujo de datos de lectura/escritura regular. Sus funciones incluyen: asignación de Regiones a los RegionServers, manejo de operaciones DDL (creación, eliminación y modificación de tablas), gestión del balanceo de carga entre RegionServers, y recuperación ante fallos de RegionServers.

RegionServers — Los Trabajadores

Los RegionServers son los nodos que realizan el trabajo real de lectura y escritura. Cada RegionServer gestiona un conjunto de Regiones (fragmentos horizontales de una tabla, divididos por rangos de RowKeys). Los componentes clave dentro de un RegionServer son:

- **MemStore:** Caché de escritura en RAM. Acumula todas las modificaciones antes de volcarlas al disco. Esta técnica convierte escrituras aleatorias (ineficientes en disco) en escrituras secuenciales (eficientes). Cuando el MemStore alcanza un umbral configurable, se vuelca (flush) al disco como un nuevo archivo HFile.
- **BlockCache:** Caché de lectura en RAM. Almacena los bloques de datos más recientemente accedidos (o más frecuentemente accedidos, según la política). Las lecturas que encuentran el dato en el BlockCache (cache hit) son extremadamente rápidas, comparables a consultas en memoria.
- **WAL (Write Ahead Log):** Log de recuperación ante desastres. Antes de escribir en el MemStore, la modificación se registra en el WAL en disco de forma secuencial (operación muy eficiente). Si el servidor falla, el WAL permite recuperar las operaciones que estaban en memoria.
- **HFiles / StoreFiles:** Los archivos inmutables en HDFS donde se persisten los datos del MemStore. Están organizados internamente como tablas hash o árboles ordenados con índices incorporados para acceso eficiente.

ZooKeeper — El Servicio de Coordinación

ZooKeeper proporciona los servicios de coordinación distribuida que necesita HBase: mantiene el registro de qué RegionServer está activo y vivo (heartbeat monitoring), almacena la ubicación del catálogo de Regiones (META table) que mapea cada rango de RowKeys al RegionServer responsable, y gestiona las elecciones de HMaster activo en configuraciones de alta disponibilidad.

4.3 Modelo de Datos Multidimensional

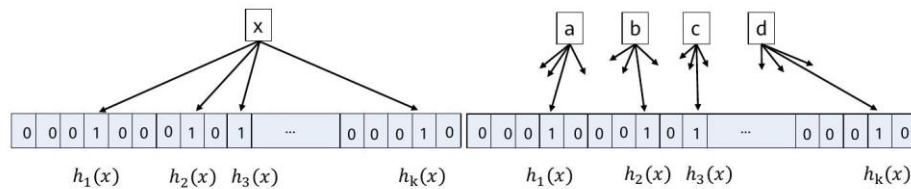
El modelo de datos de HBase difiere fundamentalmente de las tablas relacionales. En lugar de filas con columnas fijas, HBase implementa un mapa ordenado multidimensional (Sparse, Distributed, Persistent, Multidimensional Sorted Map). Cada dato se localiza mediante cuatro coordenadas:

- **RowKey (Clave de Fila):** El identificador único de cada registro. Es el parámetro más crítico del diseño: HBase clasifica y ordena todos los datos lexicográficamente en disco basándose únicamente en este campo. La proximidad lexicográfica determina la proximidad física en disco.
- **Column Family (Familia de Columnas):** Agrupación física y lógica de columnas. Todos los datos de una Column Family se almacenan en el mismo archivo HFile. Se definen al crear la tabla y no pueden cambiarse sin recrearla. Ejemplo: 'info' para datos personales, 'txn' para historial de transacciones.

- Column Qualifier (Calificador de Columna): El nombre específico de la columna dentro de una Column Family. Se crea dinámicamente al insertar datos, sin necesidad de declaración previa. Ejemplo: 'info:nombre', 'info:ciudad'.
- Timestamp (Marca de Tiempo): Permite el versionado automático. Cada celda puede almacenar múltiples valores correspondientes a momentos distintos, diferenciados por timestamp. HBase retiene automáticamente las últimas N versiones según la configuración del Column Family.

HBase BloomFilter

- BloomFilter is used to optimize some random read scenarios, that is, the **Get** scenario. It can be used to quickly determine whether a piece of data exists in a large data set.
- The BloomFilter has possibility of misjudgment when determining whether a piece of data exists. However, the judgment result of "The data does not exist" is reliable.
- HBase BloomFilter data is stored in HFiles.



Page 23 Copyright © 2020 Huawei Technologies Co., Ltd. All rights reserved.



Figura 5. Modelo de almacenamiento KeyValue en HBase mostrando la estructura multidimensional con RowKey, Column Family, Qualifier, Timestamp y Value. Fuente: Capítulo 3, Huawei Technologies (2020).

4.4 Diseño del RowKey

El diseño del RowKey es la decisión de diseño más crítica en HBase y determina, en gran medida, el rendimiento del sistema. Un mal diseño puede provocar hotspotting (toda la carga concentrada en uno o pocos RegionServers) o full table scans innecesarios. Los principios fundamentales son:

- Unicidad y Completitud: El RowKey debe identificar unívocamente cada registro. En muchos casos se construye como concatenación de múltiples atributos.
- Distribución Uniforme: Los RowKeys deben distribuirse uniformemente para evitar hotspotting. Técnicas comunes: salted keys (prefijo aleatorio o hash), reversed timestamps (para series temporales con acceso a datos recientes), hashing del identificador natural.
- Co-localización de datos relacionados: Los registros que frecuentemente se consultan juntos deben tener RowKeys similares (mismo prefijo) para que residan en la misma Región y puedan recuperarse con un solo scan de rango.
- Longitud apropiada: RowKeys más cortos reducen el overhead de almacenamiento (el RowKey se almacena en cada KeyValue). Longitudes típicas: 8-64 bytes.

Un ejemplo en el contexto financiero: para una tabla de transacciones bancarias donde las consultas más frecuentes son por cuenta y rango de fechas, un RowKey bien diseñado podría

ser: [hash(account_id, 2 bytes)] + [account_id] + [reversed_timestamp]. El hash distribuye uniformemente entre regiones, el account_id co-localiza todas las transacciones de una cuenta, y el reversed timestamp coloca las transacciones más recientes primero.

4.5 Mecanismos de Rendimiento: Compaction y Split

HBase implementa dos procesos asíncronos fundamentales para mantener el rendimiento a largo plazo: Compaction (fusión de archivos) y Region Split (división de regiones).

LSM-Tree y el Proceso de Flush

HBase implementa la estructura de datos LSM-Tree (Log-Structured Merge-Tree). El flujo es: escritura en WAL (secuencial) → escritura en MemStore (RAM) → cuando el MemStore alcanza su límite, se vuelca (flush) al disco como un nuevo HFile inmutable. Este proceso convierte todas las escrituras en operaciones secuenciales, que son 10-100x más rápidas que las aleatorias en discos magnéticos.

Compaction

A medida que se generan múltiples HFiles por flushes sucesivos, el número de archivos por Region crece, degradando el rendimiento de lectura (cada consulta debe revisar múltiples archivos). La compactación resuelve este problema:

- Minor Compaction: Fusiona un pequeño número de HFiles adyacentes (los más recientes) en un único archivo más grande. Proceso frecuente y ligero.
- Major Compaction: Fusiona TODOS los HFiles de una Region en uno solo. Elimina físicamente las celdas marcadas como borradas (tombstones) y las versiones antiguas que superan el límite configurado. Proceso costoso, generalmente programado en horas de baja actividad.

Region Split

Cuando una Region (rango de datos) crece más allá del umbral configurado (típicamente 10-20 GB), HBase la divide automáticamente por la mitad y redistribuye las dos nuevas regiones entre los RegionServers del clúster. Este proceso habilita la escalabilidad horizontal dinámica: a medida que los datos crecen, el sistema se expande automáticamente.

5. APACHE ELASTICSEARCH — MOTOR DE BÚSQUEDA COMPLEJA

5.1 Introducción y Características

Apache Elasticsearch es un motor de búsqueda y análisis distribuido, RESTful, de código abierto, construido sobre la biblioteca Apache Lucene. Fue creado por Shay Banon en 2010 y se ha convertido en el motor de búsqueda empresarial más popular del mundo, con adopción

masiva en sectores como ciberseguridad (análisis de logs con la pila ELK), e-commerce, servicios financieros y plataformas de observabilidad.

Las capacidades principales de Elasticsearch son:

- Búsqueda de texto completo (Full-Text Search): Análisis lingüístico, stemming, sinónimos, búsqueda por relevancia con puntuación TF-IDF o BM25.
- Búsqueda difusa (Fuzzy Search): Tolerancia a errores tipográficos mediante distancia de edición (Levenshtein). Buscar 'Elecsearch' puede encontrar 'Elasticsearch'.
- Autocompletado y sugerencias: Índices de tipo completion para sugerencias en tiempo real con latencias < 10 ms.
- Análisis geoespacial: Consultas por proximidad geográfica, polígonos y bounding boxes sobre coordenadas GPS.
- Análisis de series temporales: Aggregations para cálculos estadísticos sobre ventanas de tiempo (date histograms, percentiles, estadísticas).
- Schema-less: No requiere definición de esquema a priori. Inferencia automática de tipos de datos al indexar documentos JSON.

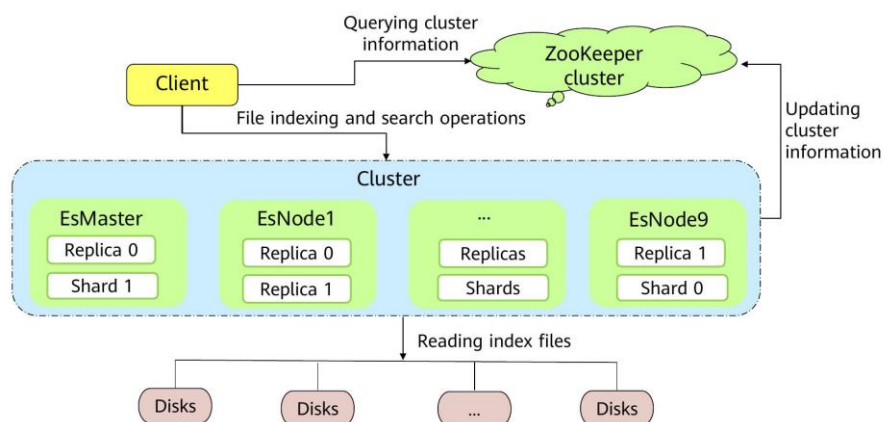
5.2 Arquitectura y Conceptos Clave

Elasticsearch organiza los datos en una jerarquía de conceptos: Clúster → Índice → Shard → Documento. Comprender esta jerarquía es fundamental para diseñar sistemas de alta disponibilidad y rendimiento.

- Clúster (Cluster): Conjunto de uno o más nodos que trabajan juntos. Identificado por un nombre único. Los nodos se descubren automáticamente mediante multicast o configuración explícita.
- Índice (Index): Equivalente lógico a una base de datos en RDBMS. Contiene documentos del mismo tipo. Un índice se distribuye en múltiples Shards para paralelismo.
- Shard (Fragmento): La unidad física de distribución. Cada shard es una instancia independiente de Lucene. Los shards primarios contienen los datos originales; los shards réplica son copias de los primarios para alta disponibilidad y escalado de lecturas.
- Documento (Document): La unidad básica de información, almacenado en JSON. Cada documento tiene un campo `_id` único dentro de su índice.

Una consideración crítica de diseño: el número de shards se fija al crear el índice y no puede modificarse posteriormente (sin reindexación). Se recomienda que el tamaño de cada shard no supere los 32 GB (tamaño máximo recomendado del heap JVM de Lucene) y que el número de shards no supere 3 veces el número de nodos del clúster para evitar overhead de coordinación.

Elasticsearch Structure

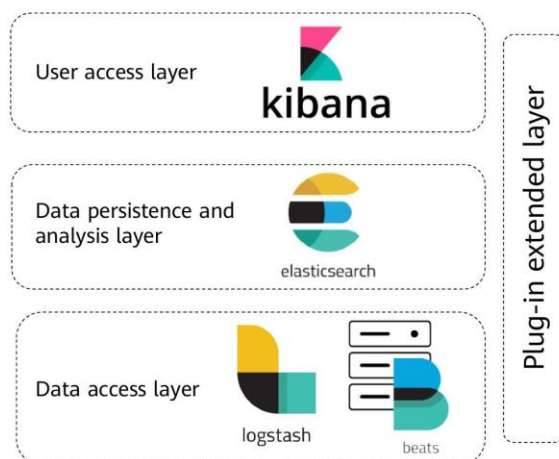


Page 46 Copyright © 2020 Huawei Technologies Co., Ltd. All rights reserved.



Figura 6. Estructura distribuida de Elasticsearch: el Cliente consulta información del clúster via ZooKeeper y realiza operaciones de indexación/búsqueda directamente. El clúster contiene EsMaster y múltiples EsNodes, cada uno con Shards primarios (P) y réplicas (R), todos leyendo archivos de índice desde sus discos locales.
Fuente: Capítulo 3, Huawei Technologies (2020).

Elasticsearch Ecosystem



- ELK and ELKB provide a complete set of solutions. They are open-source software and work together to meet the requirements of many scenarios.

Page 44 Copyright © 2020 Huawei Technologies Co., Ltd. All rights reserved.



Figura 7. Ecosistema Elasticsearch (ELK/ELKB): capa de acceso a usuario (Kibana), capa de persistencia y análisis (Elasticsearch), capa de acceso a datos (Logstash + Beats). Los plug-ins extienden las funcionalidades de cada capa. Fuente: Capítulo 3, Huawei Technologies (2020).



Elasticsearch Internal Architecture

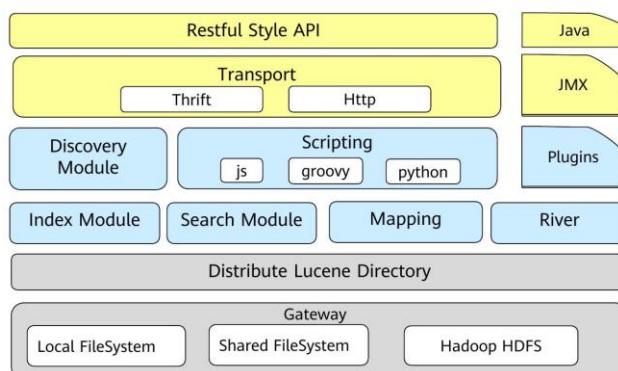


Figura 8. Arquitectura interna de Elasticsearch: capa superior de API RESTful y transporte (HTTP/Thrift), módulo de descubrimiento de nodos y scripting, módulos de indexación, búsqueda, mapping y plugins, motor de directorio Lucene distribuido, y capa de gateway con soporte para sistema de archivos local, compartido y HDFS. Fuente: Capítulo 3, Huawei Technologies (2020).

5.3 El Índice Invertido (Inverted Index)

El índice invertido (Inverted Index) es la estructura de datos central que hace posible la búsqueda de texto completo en milisegundos sobre millones de documentos. Su denominación deriva de la inversión de la perspectiva tradicional: en lugar de mapear documentos a las palabras que contienen (como un libro de texto), mapea palabras (términos) a los documentos en los que aparecen (como el índice alfabético al final de un libro de referencia).

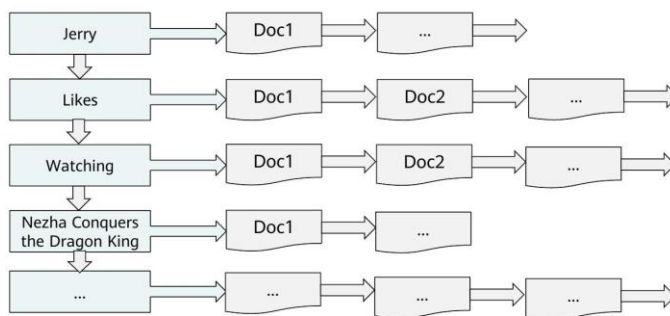
La estructura de un índice invertido contiene: un diccionario de términos (con metadatos como frecuencia de documentos), y para cada término, una posting list (lista de publicaciones) que registra los IDs de documentos donde aparece el término, su frecuencia de aparición en cada documento, y las posiciones exactas dentro del documento (para búsquedas de frases exactas).

El rendimiento excepcional del índice invertido proviene de varias optimizaciones de implementación en Lucene: el diccionario se implementa como un FST (Finite State Transducer) comprimido que reside en memoria RAM; las posting lists se comprimen mediante codificación delta y Variable-Length Encoding (VByte); y las operaciones de intersección y unión de posting lists se implementan con algoritmos de tipo merge sort sobre listas ordenadas, de complejidad $O(n)$.



Elasticsearch Inverted Indexing - Effect Diagram

- The reverse index search is to search for the corresponding document number through the keyword, and then search for the document through the document number. This is similar to look up a dictionary or searching for the content of the specified page through the book catalog.



Page 55 Copyright © 2020 Huawei Technologies Co., Ltd. All rights reserved.



Figura 9. Diagrama del efecto del Índice Invertido en Elasticsearch: cada término (Jerry, Likes, Watching, Nezha...) apunta a una posting list con los IDs de documentos donde aparece (Doc1, Doc2, ...). La búsqueda inversa encuentra el número de documento a partir de la palabra clave y luego recupera el documento completo.
Fuente: Capítulo 3, Huawei Technologies (2020).

5.4 Análisis de Texto y Tokenización

Antes de indexar un texto, Elasticsearch aplica una cadena de análisis (analysis pipeline) que transforma el texto original en los términos que se almacenarán en el índice invertido. Este proceso determina cómo se realizan las búsquedas:

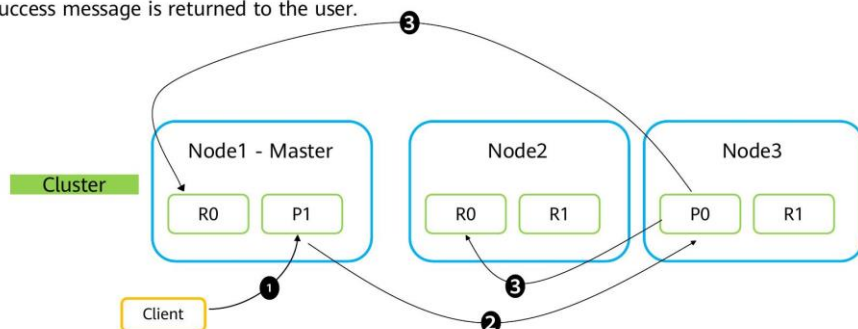
- Character Filters: Pre-procesamiento del texto. Ejemplos: remover etiquetas HTML, convertir emojis a texto, normalizar caracteres especiales.
- Tokenizer: Divide el texto en tokens (unidades atómicas). El tokenizer estándar divide por espacios y puntuación. Ejemplo: 'El Big Data es rápido.' → ['El', 'Big', 'Data', 'es', 'rápido'].
- Token Filters: Transforman la lista de tokens. Ejemplos: lowercase ('Big' → 'big'), stop words removal (eliminar 'el', 'es', 'la'), stemming ('rápido' → 'rapid'), synonyms ('rápido' → 'veloz').

El resultado final del análisis de 'El Big Data es rápido.' sería: ['big', 'data', 'rapid']. Este conjunto de términos es lo que se almacena en el índice invertido. Cuando un usuario busca 'Big Data rápido', se aplica el mismo análisis a la consulta antes de buscar en el índice, garantizando la compatibilidad entre la indexación y la búsqueda.



Elasticsearch Indexing Process

- After the client sends an index request to any node, the node determines the index shard location and forwards the request to the corresponding shard node. The shard node executes the request and sends the request to other nodes in parallel. After all the other nodes execute the request successfully, a success message is returned to the user.



Page 56 Copyright © 2020 Huawei Technologies Co., Ltd. All rights reserved.



Figura 10. Proceso de indexación en Elasticsearch: el Cliente envía una solicitud al nodo maestro (Node1-Master) que determina el shard primario destino; el shard primario ejecuta la operación y la replica en paralelo a los shards réplica de otros nodos; el éxito se confirma cuando todos los nodos completan la replicación. Fuente: Capítulo 3, Huawei Technologies (2020).

5.5 Optimización del Rendimiento en Elasticsearch

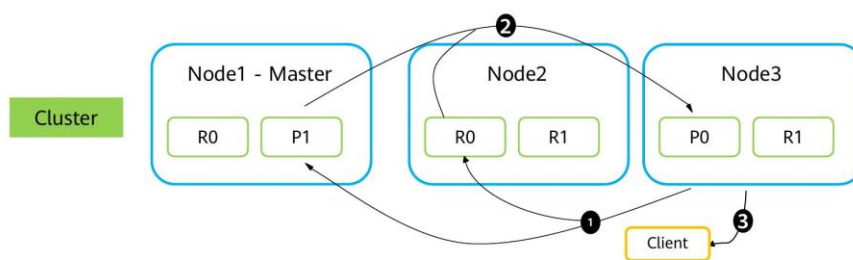
Existen varios parámetros y estrategias para optimizar el rendimiento de Elasticsearch en entornos Big Data:

- Política de Shards y Réplicas: El número de shards debe calcularse en base al volumen de datos esperado y el número de nodos. Una réplica (1 copia adicional) es generalmente suficiente; más réplicas aumentan el overhead de almacenamiento y escritura.
- Optimización de Bulk Indexing: Para cargas masivas de datos, usar la API Bulk con lotes de 5-15 MB. Aumentar el tamaño del índice de actualización (index.refresh_interval) durante la ingesta masiva.
- Balanceo rápido post-expansión: Tras añadir nodos, ajustar los parámetros cluster.routing.allocation.cluster_concurrent_rebalance y indices.recovery.max_bytes_per_sec para acelerar el rebalanceo.
- Gestión de memoria: El heap JVM debe ser al menos el 50% de la RAM disponible, pero no más de 32 GB (para evitar que el GC de Java sea ineficiente con espacios de memoria grandes).



Elasticsearch Search Process

- After receiving all the data to be retrieved, the node sends a request to the data-related shard. The shard that receives the request reads the data and returns the data to the retrieval node. The retrieval node summarizes the result and returns the result to the client.



Page 58 Copyright © 2020 Huawei Technologies Co., Ltd. All rights reserved.



Figura 11. Proceso de búsqueda en Elasticsearch: el Cliente envía la consulta a cualquier nodo del clúster (paso 1); ese nodo reenvía sub-consultas a los shards relevantes de todos los nodos (paso 2); cada shard responde con sus resultados locales; el nodo coordinador fusiona y devuelve el resultado final al cliente (paso 3). Fuente: Capítulo 3, Huawei Technologies (2020).

6. ARQUITECTURA COMBINADA: HBASE + ELASTICSEARCH

6.1 Secondary Indexing

El patrón de Secondary Indexing (Indexación Secundaria) es la solución arquitectónica que supera la principal limitación de HBase: su dependencia del RowKey para búsquedas eficientes. HBase es extraordinariamente rápido cuando se busca por RowKey exacto o rango de RowKeys, pero realizar un filtrado por atributos no incluidos en el RowKey (búsqueda por ciudad, ocupación, rango de importes, texto libre) requeriría un Full Table Scan, operación equivalente al procesamiento batch y totalmente incompatible con los requisitos de tiempo real.

La solución es crear un índice secundario en Elasticsearch: al momento de ingestar datos en HBase, un proceso paralelo indexa los metadatos y atributos consultables en Elasticsearch, asociando cada documento indexado con el RowKey de HBase correspondiente. Elasticsearch actúa así como un índice invertido sobre los atributos de HBase, respondiendo la pregunta '¿cuáles son los RowKeys de los registros que cumplen estos criterios?' de forma ultrarrápida.

6.2 Flujo de Ingesta y Consulta

El flujo arquitectónico completo de la solución combinada HBase + Elasticsearch puede describirse en dos sub-flujos: el flujo de escritura (ingesta) y el flujo de lectura (consulta).

Flujo de Escritura (Ingesta)

- Paso 1 — Ingesta de datos crudos: Los datos (imágenes, textos, registros transaccionales) llegan al sistema vía Apache Kafka u otro sistema de mensajería.
- Paso 2 — Almacenamiento en HBase: Los datos completos se escriben en HBase con un RowKey diseñado para distribución uniforme.
- Paso 3 — Indexación paralela en Elasticsearch: Un proceso paralelo (típicamente implementado como un HBase Coprocessor, un job de Apache Flink, o un proceso Kafka Consumer) extrae los metadatos y atributos clave del registro y los indexa en Elasticsearch, incluyendo el RowKey de HBase como campo especial.

Flujo de Lectura (Consulta)

- Paso 4 — Consulta compleja del usuario: El usuario formula una consulta de múltiples atributos, como 'Transacciones fallidas en Lima en las últimas 24 horas por montos superiores a S/ 5,000'.
- Paso 5 — Resolución en Elasticsearch: La capa de aplicación envía la consulta a Elasticsearch, que la resuelve ultrarrápidamente usando su índice invertido y devuelve una lista de RowKeys de HBase correspondientes.
- Paso 6 — Recuperación masiva en HBase: La aplicación toma los RowKeys devueltos por Elasticsearch y ejecuta un Multi-Get en HBase para recuperar los registros completos en milisegundos.
- Paso 7 — Respuesta al usuario: Los datos completos se presentan al usuario con una latencia total que combina el tiempo de Elasticsearch (100-300 ms) más el tiempo de HBase (< 100 ms por registro).

Esta arquitectura representa el estándar de oro de la industria para Real-Time Retrieval en Big Data. Empresas como Alibaba, Huawei, JD.com y múltiples instituciones financieras globales implementan variantes de este patrón para gestionar sus plataformas de datos a escala de Petabytes.

7. GES — GRAPH ENGINE SERVICE

7.1 Introducción al Análisis de Grafos en Big Data

Muchos problemas del mundo real son inherentemente grafales: las relaciones entre entidades son tan importantes como los atributos de las entidades individuales. La detección de redes de fraude bancario requiere identificar patrones de transacciones entre miles de cuentas relacionadas. El análisis de redes sociales requiere calcular comunidades e influenciadores en grafos de miles de millones de conexiones. La optimización de rutas logísticas requiere encontrar caminos mínimos en grafos de millones de nodos.

Las bases de datos relacionales son extremadamente ineficientes para consultas grafales: cada nivel de relación requiere un JOIN adicional, y los JOINS de múltiples niveles (multi-hop

traversal) escalan exponencialmente en tiempo. Las bases de datos de grafos en memoria (Neo4j, JanusGraph) son eficientes pero no escalan a grafos de miles de millones de nodos.

GES (Graph Engine Service) de Huawei es una base de datos de grafos distribuida que resuelve ambos problemas: es eficiente para consultas grafales complejas Y escala a grafos masivos, al construirse sobre HBase (para almacenamiento distribuido) y Elasticsearch (para búsquedas en los atributos de nodos y aristas).

7.2 Arquitectura de GES

GES implementa el modelo de Property Graph (grafo de propiedades), donde tanto los nodos (vértices) como las aristas (relaciones) pueden tener atributos arbitrarios. La arquitectura se organiza en capas:

- Capa de Interfaz: Proporciona dos APIs principales. La Gremlin API, compatible con el estándar Apache TinkerPop, permite expresar consultas de traversal de grafos con el lenguaje Gremlin (e.g., `g.V().has('ciudad','Lima').out('CLIENTE_DE').in('FRAUDE')`). La REST API proporciona endpoints para operaciones CRUD, algoritmos de análisis y gestión del grafo.
- Capa de Cómputo: El motor principal de GES gestiona el almacenamiento de metadatos, la ejecución de algoritmos de grafos (PageRank, detección de comunidades, cálculo de caminos mínimos), y la coordinación entre el almacenamiento y las interfaces.
- Capa de Almacenamiento: Almacenamiento KV distribuido basado en HBase para los datos masivos del grafo. Elasticsearch para búsqueda en atributos de nodos y aristas (búsqueda difusa de entidades).

Los casos de uso más importantes de GES en el contexto financiero y empresarial incluyen:

- Anti-lavado de dinero (AML): Detección de redes circulares de transacciones que fragmentan grandes montos en pequeñas operaciones a través de múltiples cuentas intermediarias.
- Scoring crediticio basado en relaciones: Un cliente sin historial propio puede evaluarse mediante la calidad de sus conexiones en el grafo financiero.
- Detección de fraude en tarjetas: Identificar cuando una tarjeta es usada en múltiples ubicaciones geográficas simultáneas (patrones imposibles en el grafo espacio-temporal).
- Análisis de redes sociales: Detección de comunidades, identificación de influenciadores, propagación de contenido viral.

8. CASOS DE USO EMPRESARIALES

8.1 Sector Financiero

El sector financiero es el caso de uso paradigmático para Real-Time Retrieval en Big Data, dado que combina volúmenes de datos masivos con requisitos de latencia extremadamente bajos y consecuencias directas en el negocio.



Industry Application - Finance

- Querying by ID (log type, card number, etc.) and time segment
- Querying transaction vouchers, tracing transactions, and querying customer credit records helps customers quickly obtain loans.
- Integrating various data, forming a relationship network, and querying association relationships can be used in scenarios such as anti-money laundering and risk control.

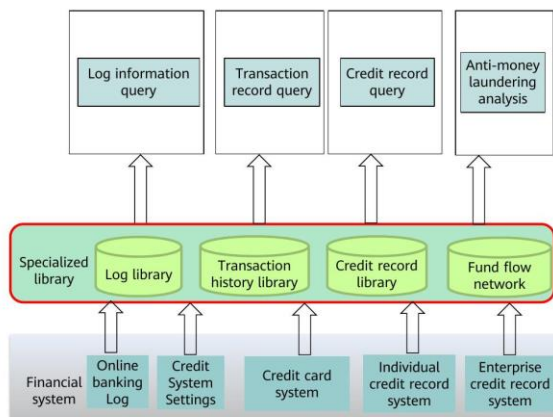


Figura 12. Aplicación de Real-Time Retrieval en el sector financiero: consultas de log por ID/tiempo, rastreo de transacciones, consulta de historial crediticio y análisis anti-lavado de dinero. El sistema integra biblioteca especializada, log, historial transaccional, historial crediticio y red de flujo de fondos. Fuente: Capítulo 3, Huawei Technologies (2020).

Sistema de Verificación de Crédito en Tiempo Real

Cuando un cliente solicita un crédito (en el momento del pago, en un cajero, o vía app), el sistema debe evaluar su perfil de riesgo en menos de 2 segundos. El proceso implementado con la arquitectura HBase + Elasticsearch es el siguiente: el perfil del cliente (historial de crédito, ingresos declarados, deudas actuales, comportamiento de pago) se almacena en HBase con el número de documento como base del RowKey. Los atributos relevantes (ciudad, segmento de riesgo, productos activos) se indexan en Elasticsearch. Al recibir la solicitud, la aplicación recupera el perfil completo de HBase en < 50 ms y aplica los modelos de scoring en memoria.

Detección de Fraude Transaccional

Cada transacción con tarjeta de crédito debe evaluarse en tiempo real antes de ser aprobada. El sistema mantiene en HBase el historial de transacciones recientes del cliente (últimas 90 días), su patrón de comportamiento (ubicaciones habituales, importes típicos, comercios frecuentados) y listas negras de comerciantes o tarjetas comprometidas. GES mantiene el grafo de relaciones entre cuentas para detectar patrones de fraude coordinado. La decisión de aprobar o rechazar debe tomarse en < 300 ms para no afectar la experiencia del usuario.

Rastreo de Transacciones Bancarias

Los auditores y el equipo de cumplimiento necesitan reconstruir el historial completo de transacciones de una cuenta, incluyendo transferencias a otras cuentas, consultas de saldo,

cambios de datos personales y accesos al sistema. HBase almacena este historial con RowKeys diseñados para co-localizar todas las operaciones de una cuenta. Elasticsearch permite búsquedas por criterios como tipo de operación, canal, importes o estado.

8.2 E-Commerce y Retail

Las plataformas de comercio electrónico requieren búsquedas de catálogo ultrarrápidas sobre millones de productos con múltiples filtros simultáneos (precio, categoría, marca, valoración, disponibilidad, ubicación del almacén).

- Búsqueda de catálogo: Elasticsearch gestiona el índice de búsqueda del catálogo, con análisis lingüístico para manejar búsquedas por nombre de producto (incluyendo errores tipográficos y sinónimos), filtros de faceted search (facetas por categoría, precio, marca) y ordenamiento por relevancia personalizada.
- Perfil de cliente en tiempo real: HBase almacena el historial de compras, el carrito actual, las preferencias y el historial de navegación. Al acceder un usuario, su perfil completo se recupera en < 50 ms para personalizar la experiencia.
- Inventario en tiempo real: HBase gestiona el stock disponible con lecturas y escrituras de alta frecuencia, evitando la sobreventa en periodos de alta demanda (campañas, Black Friday).

8.3 Ciberseguridad y Análisis de Logs

Los sistemas de SIEM (Security Information and Event Management) procesan millones de eventos de seguridad por segundo, generados por firewalls, IDS/IPS, servidores, endpoints y aplicaciones. La pila ELK (Elasticsearch, Logstash, Kibana) se ha convertido en el estándar de la industria para este caso de uso.

- Ingesta y enriquecimiento: Logstash o Apache Flume colectan logs de múltiples fuentes, los parsean, normalizan y enriquecen (geolocalización de IPs, lookup de dominios, extracción de User-Agents) antes de indexarlos en Elasticsearch.
- Búsqueda de amenazas (Threat Hunting): Los analistas de seguridad usan Elasticsearch para buscar patrones de ataque en el historial de eventos: intentos de autenticación fallidos, accesos a rutas sensibles, comunicaciones con IPs maliciosas conocidas.
- Correlación de eventos: Elasticsearch Aggregations permite correlacionar eventos de múltiples fuentes para detectar ataques complejos que se manifiestan como anomalías distribuidas en el tiempo.

8.4 Telco y Call Centers (Customer 360)

Las compañías de telecomunicaciones gestionan decenas o cientos de millones de clientes y miles de millones de registros de llamadas (CDRs — Call Detail Records), mensajes y eventos de red. El caso de uso Customer 360 ejemplifica el potencial de Real-Time Retrieval.

Cuando un cliente llama al servicio de atención, el operador necesita visualizar instantáneamente el historial completo del cliente: plan contratado, últimas facturas, historial

de llamadas recientes, reclamos pendientes, historial de incidencias técnicas y estado de la red en la zona del cliente. Este perfil 360° se construye recuperando datos de múltiples sistemas en paralelo usando HBase (para datos estructurados de alta frecuencia como CDRs) y Elasticsearch (para búsqueda por atributos como tipo de incidencia o zona geográfica). La latencia objetivo es < 1 segundo desde que el operador recibe la llamada hasta que el perfil completo está visible en pantalla.

Adicionalmente, GES permite análisis anti-lavado de dinero a través de la red de comunicaciones: identificar grupos de números que se llaman mutuamente en patrones sospechosos, o detectar redes de fraude de telecomunicaciones (SIM swapping, robo de identidad telefónica).

9. CONCLUSIONES

- La Sinergia Tecnológica es el Estándar de Oro: La recuperación en tiempo real en ecosistemas Big Data modernos no depende de una sola herramienta, sino de la combinación estratégica de múltiples motores especializados. HBase + Elasticsearch representa el patrón arquitectónico más adoptado en la industria, donde HBase proporciona el almacenamiento masivo de datos crudos y Elasticsearch proporciona la indexación ágil por múltiples atributos (Secondary Indexing). Esta sinergia permite responder a preguntas complejas de negocio que ningún componente individual podría resolver de forma eficiente.
- El Diseño Arquitectónico Determina el Rendimiento: La latencia de milisegundos no es resultado de hardware más rápido, sino de decisiones de diseño precisas. El RowKey en HBase debe diseñarse para: (a) distribución uniforme evitando hotspotting, (b) co-localización de datos frecuentemente consultados juntos, y (c) soporte directo para los patrones de acceso más frecuentes. El esquema de partición (sharding) en Elasticsearch debe garantizar una distribución equilibrada de la carga entre nodos.
- La Memoria RAM es el Habilitador Fundamental: El uso intensivo de la RAM —MemStore y BlockCache en HBase, índice invertido en memoria en Elasticsearch— transforma el histórico cuello de botella del disco magnético. Las arquitecturas modernas de Real-Time Retrieval son esencialmente arquitecturas de acceso a datos en memoria con persistencia en disco, no al revés.
- La Estructura de Datos Supera al Hardware: La estructura LSM-Tree de HBase convierte escrituras aleatorias (lentas) en secuenciales (rápidas). El índice invertido de Elasticsearch convierte búsquedas de texto (imposibles de optimizar con B-Trees) en intersecciones de posting lists (complejidad lineal). En ambos casos, la elección de la estructura de datos correcta supera en varios órdenes de magnitud cualquier mejora de hardware.
- GES Amplía el Paradigma hacia Datos Relacionales Complejos: Para problemas donde las relaciones entre entidades son tan importantes como los atributos individuales (detección de fraude en red, análisis de influencia, optimización de rutas), las bases de datos de grafos distribuidas como GES complementan a HBase y Elasticsearch, completando el ecosistema de Real-Time Retrieval.
- Transición de Paradigmas y Relevancia Actual: La adopción masiva de estas tecnologías por parte de líderes globales como Alibaba, Amazon, Google, Facebook y Huawei valida que el paradigma de Real-Time Retrieval con arquitecturas NoSQL distribuidas no es una tendencia emergente, sino la norma establecida para cualquier sistema de datos que opere a escala. Las organizaciones que aún dependen exclusivamente de RDBMS para cargas de trabajo a escala de Petabytes enfrentan limitaciones fundamentales que no pueden resolverse con mejoras incrementales del hardware.

10. REFERENCIAS BIBLIOGRÁFICAS

- [1] Huawei Technologies Co., Ltd. (2022). *Big Data Scenario-based Solutions: Real-Time Retrieval* (Capítulo 3). Huawei ICT Academy – HCIP-Big Data Developer V2.0.
- [2] Huawei Technologies Co., Ltd. (2022). *HCIP-Big Data Developer V2.0 Training Material*. Material de certificación no publicado. Huawei Technologies.
- [3] Apache HBase Team. (2024). *Apache HBase Reference Guide*. Apache Software Foundation.
<https://hbase.apache.org/book.html>
- [4] Apache Lucene Team. (2024). *Apache Lucene Core Documentation*. Apache Software Foundation.
<https://lucene.apache.org/core/>
- [5] Elastic N.V. (2024). *Elasticsearch: The Official Distributed Search & Analytics Engine*. Elasticsearch Documentation. <https://www.elastic.co/guide/en/elasticsearch/reference/>